

PIXEL MEMORIES — MASTER DOCUMENT

문서 목적

이 문서는 Pixel Memories 프로젝트의 **기획·설계·개발 방향을 유지하기 위한 기준 문서이자 개발 인수 인계 문서**다.

문서는 크게 두 부분으로 나눈다.

PART A — PROJECT BLUEPRINT - 프로젝트의 정체성 - 기획 - 기능 - 구조 - 기술 방향 - MVP 로드맵

PART B — DEVELOPMENT HANDOFF - 현재 실제 구현 상태 - 개발환경 - 최근 작업 - 다음 작업 - 주의사항

개발을 이어갈 때 **PART A**는 큰 방향이 변경되지 않는 한 유지하고,
PART B만 현재 개발 상태에 맞게 지속적으로 갱신한다.

PART A. PROJECT BLUEPRINT

1. 프로젝트 개요

프로젝트명

Pixel Memories

한 줄 정의

추억을 탐험하는 픽셀 RPG형 모바일 청첩장

기존 모바일 청첩장처럼 사진과 텍스트를 위에서 아래로 읽는 것이 아니라, 하객이 작은 픽셀 공간을 직접 돌아다니며 신랑·신부의 추억과 결혼식 정보를 발견하는 서비스다.

2. 프로젝트가 해결하려는 문제

현재 모바일 청첩장은 대부분 비슷한 구조를 가진다.

메인 사진
↓
인사말
↓
신랑 / 신부 소개

↓
사진
↓
예약 정보
↓
지도
↓
계좌번호

디자인은 달라도 사용자 경험은 대부분 스크롤해서 정보를 읽는 것에 머문다.

Pixel Memories는 이를:

읽는 청첩장
→
탐험하는 청첩장

으로 바꾸는 것을 목표로 한다.

3. 핵심 경험

하객은 청첩장을 단순히 읽는 것이 아니라 작은 추억의 공간에 초대된다.

예:

[결혼식장]
|
[사진관] — 광장 — [카페]
|
[추억의 집]
|
[정원]

하객 캐릭터가 공간을 돌아다닌다.

각 장소에는 신랑·신부의 기억이 배치된다.

예:

사진 액자
→ 실제 사진 보기

NPC

→ 해당 시기의 이야기

게시판

→ 방명록

앨범

→ 사진 모음

결혼식장

→ 날짜 / 시간 / 장소

버스 정류장

→ 오시는 길

우체통

→ 축하 메시지

따라서 핵심 경험은:

이동 → 발견 → 상호작용 → 추억 확인

이다.

4. 핵심 디자인 원칙

4.1 게임이 아니라 경험형 웹이다

Pixel Memories는 본격적인 RPG를 만드는 프로젝트가 아니다.

게임 요소는 콘텐츠를 탐색하기 위한 인터페이스다.

따라서 복잡한:

- 전투
- 레벨
- 아이템 파밍
- 퀘스트 시스템

등은 핵심 범위가 아니다.

핵심은:

걷는다

↓

발견한다
↓
누른다
↓
추억을 본다

이다.

4.2 모바일 우선

최종 사용자인 하객은 대부분 카카오톡 등의 링크를 통해 스마트폰으로 접속할 가능성이 높다.

따라서:

Mobile Web First

를 기본 원칙으로 한다.

앱 설치를 요구하지 않는다.

카카오톡 링크
↓
브라우저
↓
Pixel Memories 입장

이 이상적인 흐름이다.

4.3 진입 장벽은 매우 낮아야 한다

게임에 익숙하지 않은 사용자도 사용할 수 있어야 한다.

따라서 복잡한 조작법을 요구하지 않는다.

예:

화면 터치
→ 캐릭터 해당 위치로 이동

또는

가상 조이스틱
→ 캐릭터 이동

최종 조작 방식은 모바일 UX 테스트 후 결정한다.

5. 사용자 유형

크게 두 종류의 사용자가 존재한다.

5.1 제작자

주로:

신랑
신부

이다.

제작자는 자신의 Pixel Memories 공간을 만든다.

향후 서비스화할 경우 제작 과정은 가능한 한 쉽게 만들어야 한다.

예:

테마 선택
↓
신랑 / 신부 정보 입력
↓
결혼식 정보 입력
↓
사진 업로드
↓
사진별 설명 작성
↓
배치할 장소 선택
↓
미리보기
↓
청첩장 링크 생성

5.2 방문자

주로:

하객
친구
가족
지인

이다.

별도 회원가입 없이 링크로 접근하는 것을 기본 방향으로 한다.

청첩장 링크
↓
공간 입장
↓
캐릭터 탐색
↓
추억 발견
↓
결혼식 정보 확인
↓
방명록 / 사진 남기기

6. 방문자 시나리오 예시

카카오톡으로 링크를 받는다.

"저희 결혼합니다 🍷"
[Pixel Memories 입장하기]

링크를 누르면 시작 화면이 나타난다.

MINA ♥ JUNHO

우리의 추억 마을에
초대합니다.

[입장하기]

입장하면 픽셀 공간이 나타난다.

캐릭터를 움직여 사진 액자로 이동한다.

캐릭터
↓
[사진 액자]

상호작용하면 React UI가 열린다.

실제 사진
"처음 제주 여행을
갔던 날"
2022.05

다시 맵으로 돌아가 다른 추억을 탐험한다.

7. 주요 콘텐츠 유형

Pixel Memories의 공간에는 다양한 인터랙션 오브젝트가 존재할 수 있다.

사진

액자
앨범
사진관
폴라로이드

상호작용하면 실제 사진과 설명을 보여준다.

NPC

특정 시기의 신랑·신부 또는 주변 인물을 표현할 수 있다.

NPC와 상호작용하면 말풍선이나 스토리가 나온다.

게시판

방명록 또는 축하 메시지 기능.

결혼식장

다음 정보를 제공한다.

날짜
시간
장소
연락처

교통 오브젝트

예:

버스 정류장
자동차
표지판

상호작용하면:

지도
주소
대중교통
주차정보

등을 보여준다.

8. 실사진과 픽셀 그래픽 문제

Pixel Memories에서 중요한 디자인 이슈 중 하나다.

픽셀 공간은 감성적이지만 실제 사진을 그대로 표시하면:

Pixel World
+
Real Photo

사이에 시각적인 이질감이 발생할 수 있다.

따라서 다음 방법을 검토한다.

방법 A — 픽셀 프레임

실사진 자체는 유지하되 픽셀 UI 프레임 안에서 표시.

방법 B — 사진 필터

색감이나 질감을 공간 테마와 맞춘다.

방법 C — 도트 변환

선택적으로 사진을 픽셀 아트 스타일로 변환한다.

단, 원본 사진도 볼 수 있도록 한다.

현재 우선 방향

MVP에서는:

원본 사진 + 픽셀 스타일 프레임

부터 테스트한다.

AI 도트 변환 등은 후순위 기능으로 둔다.

9. 애니메이션

Pixel Memories의 감성을 결정하는 중요한 요소다.

정적인 화면만 존재하면 단순한 픽셀 테마 웹페이지처럼 느껴질 수 있다.

따라서 최소한:

캐릭터 걷기
NPC idle animation
나무 흔들림
물결

불빛
간단한 파티클

등의 애니메이션을 활용한다.

다만 MVP에서는 가장 먼저:

플레이어 이동 애니메이션

부터 구현한다.

10. MVP 정의

MVP의 목적은:

이 컨셉이 실제 모바일 웹에서 재미있는가?

를 검증하는 것이다.

처음부터 청첩장 제작 플랫폼 전체를 만들지 않는다.

MVP 1 — 탐험 가능 여부

목표:

브라우저에서 작은 픽셀 공간을 돌아다닐 수 있다.

필수:

맵 표시
플레이어 표시
캐릭터 이동
카메라
모바일 대응

MVP 2 — 추억 탐색

목표:

공간을 돌아다니면서 실제 추억을 볼 수 있다.

추가:

상호작용 오브젝트
사진 모달
사진 설명
NPC 대화

MVP 3 — 청첩장 기능

추가:

신랑 / 신부 소개
예식 날짜
예식 장소
지도
연락처

이 단계가 되면:

실제로 사용할 수 있는 RPG형 모바일 청첩장

이 된다.

MVP 4 — 참여 기능

추가:

방명록
축하 메시지
하객 사진 업로드

11. MVP 이후 확장 아이디어

청첩장을 첫 번째 사용 사례로 삼되 구조적으로는 더 확장할 수 있다.

```
Pixel Memories
|
├─ Wedding
|
├─ Couple Memory
|
```

```

├─ Birthday
|
├─ Travel Memory
|
└─ Personal Minihome

```

장기적으로는:

나만의 2D 추억 공간

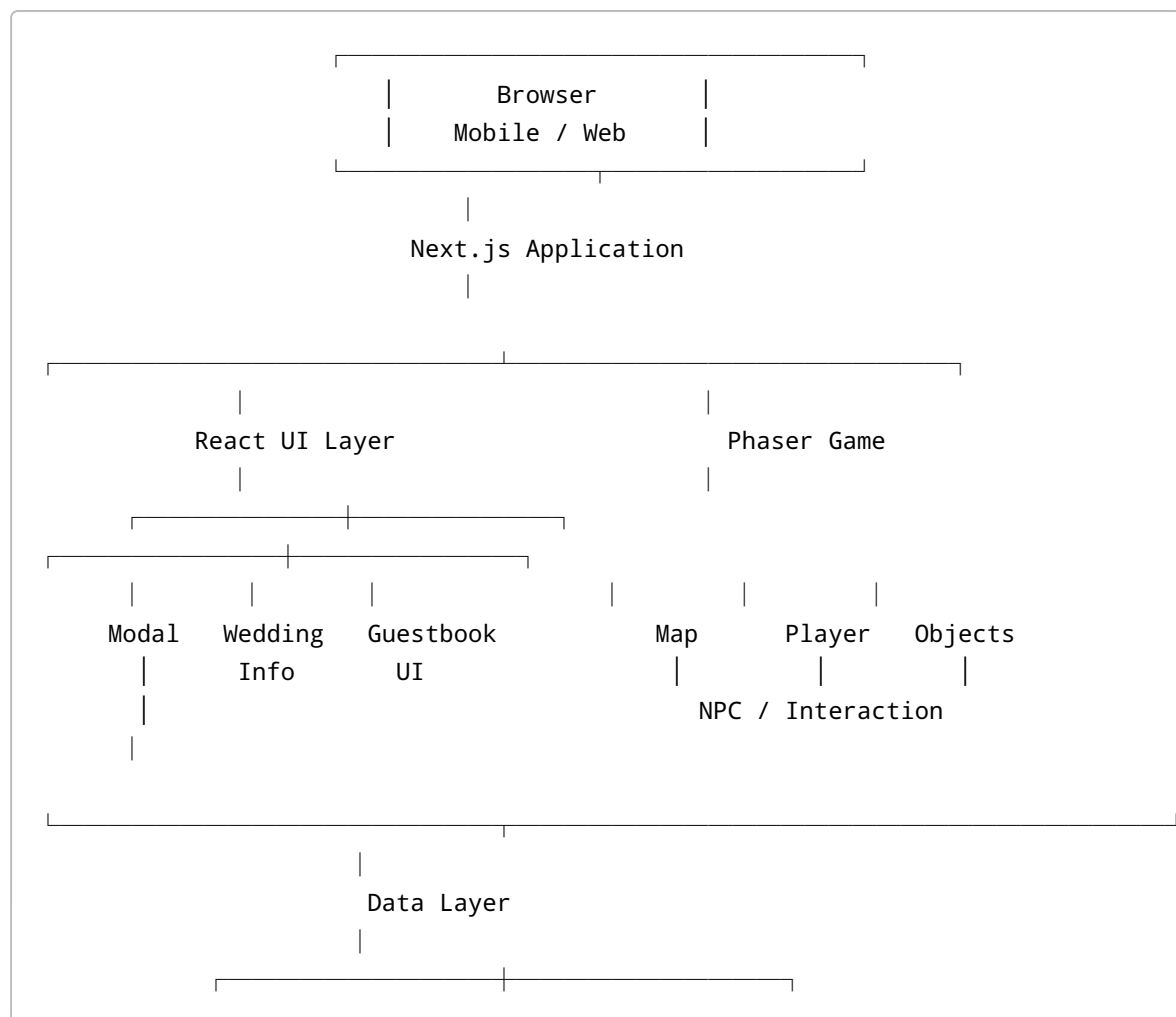
이라는 방향으로 확장 가능하다.

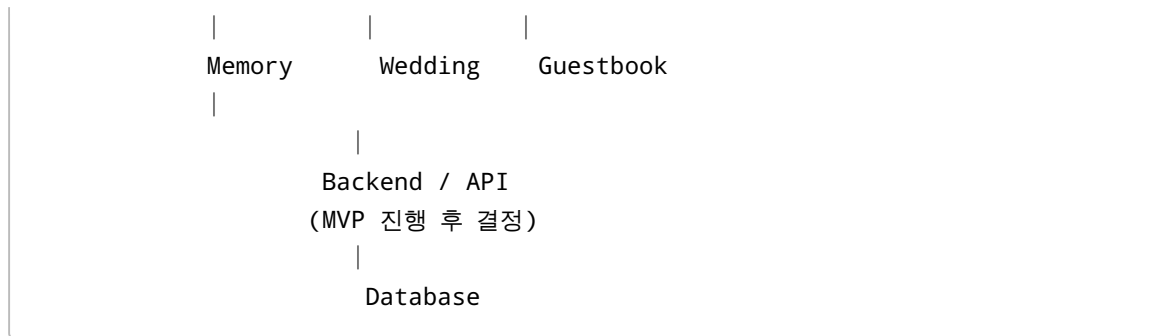
과거 미니홈피의 감성과 공간형 인터페이스를 결합한 형태도 고려한다.

단, 현재 개발에서는 이 확장을 구현하지 않는다.

12. 전체 시스템 구조 초안

현재 생각하는 대략적인 구조는 다음과 같다.





현재 핵심 분리는:

Next.js / React
= 웹 UI와 서비스 화면

Phaser
= 픽셀 공간과 캐릭터 상호작용

이다.

13. React와 Phaser의 역할

둘의 역할을 명확히 구분한다.

Phaser

게임 공간 담당.

맵
캐릭터
이동
충돌
NPC
오브젝트
카메라
애니메이션

React / Next.js

일반 웹 UI 담당.

사진 모달
결혼식 정보
방명록
설정
제작 화면
로그인
폼

예:

Phaser

캐릭터가 액자에 접근
↓
상호작용 발생
↓
React에게 이벤트 전달
↓
React

사진 모달 표시

모든 UI를 Phaser 안에서 구현하지 않는다.

14. 프론트엔드 구조 초안

향후 실제 구현 과정에서 변경될 수 있다.

```
src 또는 프로젝트 루트
|
├─ app/
|   ├─ page.tsx
|   ├─ layout.tsx
|   └─ ...
|
├─ components/
|   ├─ game/
|   │   └─ GameCanvas.tsx
|   │
|   ├─ memory/
|   │   └─ MemoryModal.tsx
|   │
|   └─ wedding/
|       └─ WeddingInfo.tsx
```

```

|   |
|   └─ guestbook/
|
|   └─ game/
|       ├── config/
|       ├── scenes/
|       ├── objects/
|       ├── player/
|       └─ maps/
|
|   └─ data/
|
|   └─ types/
|
|   └─ public/
|       ├── sprites/
|       ├── maps/
|       ├── images/
|       └─ audio/

```

아직 이 구조를 실제로 만들지는 않았다.

필요해지는 시점에 점진적으로 만든다.

15. Phaser 내부 구조 예상

초기에는 복잡하게 나누지 않는다.

첫 구조는:

```

Phaser Game
|
└─ BootScene
|
└─ MainScene
    ├── Map
    ├── Player
    ├── Camera
    └─ Interaction

```

정도로 시작한다.

프로젝트가 커지면:

```
BootScene
PreloadScene
WorldScene
WeddingScene
```

등으로 분리할 수 있다.

16. 데이터 모델 초안

아직 DB를 만들지 않는다.

다만 어떤 데이터가 필요한지는 미리 생각한다.

Wedding

```
Wedding
- id
- coupleName
- weddingDate
- venue
- address
- latitude
- longitude
```

Memory

```
Memory
- id
- title
- description
- imageUrl
- date
- objectId
- position
```

Map Object

```
MapObject
- id
- type
- x
- y
```



```
- interactionType
- targetId
```

예:

```
액자 #3
↓
interactionType = MEMORY
targetId = memory_003
```

Guestbook

```
Guestbook
- id
- author
- message
- createdAt
```

실제 구현 시 요구사항에 맞게 수정한다.

17. 기술 스택

현재 확정

```
Next.js
React
TypeScript
Tailwind CSS
Git
GitHub
```

다음 도입 예정

```
Phaser.js
```

향후 검토

```
Backend
Database
Image Storage
```

Authentication
Deployment

Spring Boot 사용 가능성은 있으나 현재 확정하지 않는다.

MVP에서 실제로 필요한 시점에 결정한다.

18. 개발 원칙

원칙 1 — 작동하는 작은 결과부터 만든다

처음부터:

회원가입
관리자
DB
방명록
업로드
게임
청첩장

전부 만들지 않는다.

가장 먼저:

캐릭터 하나가 움직이는 웹페이지

를 만든다.

원칙 2 — 기능 단위로 완성한다

예:

Canvas 표시
↓
commit

Player 표시
↓
commit

Player 이동
↓

commit

Map 표시



commit

Interaction



commit

원칙 3 — AI가 코딩하더라도 구조를 이해한다

사용자는 React / Next.js / Phaser / Git을 처음 사용한다.

따라서 AI가 코드를 작성하더라도 최소한 다음은 이해하면서 진행한다.

이 파일은 왜 존재하는가?

브라우저에서 무엇이 실행되는가?

Next.js는 무엇을 담당하는가?

React는 무엇을 담당하는가?

Phaser는 무엇을 담당하는가?

npm 명령은 무엇을 실행하는가?

Git은 지금 무엇을 기록하는가?

원칙 4 — 과도한 선행학습을 하지 않는다

React 전체 강의를 끝내고 개발을 시작하는 방식이 아니다.

필요한 기능 등장



관련 개념 학습



직접 구현



다음 기능

방식으로 진행한다.

19. 개발 로드맵

현재 예상 순서:

PHASE 0

프로젝트 환경 구축

✓ Next.js

✓ Git

✓ GitHub

↓

PHASE 1

Phaser 연결

↓

PHASE 2

Canvas 표시

↓

PHASE 3

Map

↓

PHASE 4

Player

↓

PHASE 5

Movement

↓

PHASE 6

Camera

↓

PHASE 7

Object Interaction



PHASE 8

Memory Modal



PHASE 9

Wedding Information



PHASE 10

Mobile UX



FIRST PLAYABLE MVP

이후:

Guestbook

Photo Upload

Creator

Backend

DB

Deployment

등으로 확장한다.

PART B. DEVELOPMENT HANDOFF

이 부분은 개발 진행에 따라 계속 갱신한다.

새 채팅으로 이동하거나 다른 AI/개발 환경에서 이어갈 경우 이 섹션을 가장 먼저 확인한다.

20. 현재 개발 상태

기준일

2026-08-18

현재 단계

PHASE 0 - 프로젝트 환경 구축 완료

다음:

PHASE 1 - Phaser 도입

현재 아직 Pixel Memories 고유 기능 구현은 시작하지 않았다.

21. 로컬 프로젝트

Windows 환경.

프로젝트 위치:

```
C:\dev\pixel-memories
```

프로젝트 생성 명령:

```
npx create-next-app@latest pixel-memories
```

생성 성공 확인 완료.

22. 현재 프로젝트 구성

현재 주요 파일:

```
pixel-memories/  
├─ app/  
│  ├─ favicon.ico  
│  ├─ globals.css  
│  ├─ layout.tsx  
│  └─ page.tsx  
├─ public/  
├─ node_modules/  
├─ .next/  
├─ .git/  
├─ .gitignore  
├─ AGENTS.md  
└─ CLAUDE.md
```

```
├ README.md
├ eslint.config.mjs
├ next.config.ts
├ package.json
├ package-lock.json
├ postcss.config.mjs
└ tsconfig.json
```

현재 아직 `components/`, `game/` 등의 Pixel Memories 전용 구조는 만들지 않았다.

23. Next.js 실행 상태

실행:

```
npm run dev
```

정상 동작 확인 완료.

브라우저:

```
http://localhost:3000
```

Next.js 기본 화면 표시 확인 완료.

참고

`npm run dev` 는 로컬 Next.js 개발 서버를 실행한다.

터미널에서:

```
Ctrl + C
```

를 누르면 서버가 종료된다.

24. Git 상태

Git 초기화 완료.

```
git init
```

기본 브랜치는:

```
main
```

으로 변경했다.

사용 명령:

```
git branch -M main
```

25. 최초 Commit

Staging:

```
git add .
```

최초 commit:

```
git commit -m "260813_PROJECT_SETUP"
```

결과:

```
[main (root-commit) de96b45] 260813_PROJECT_SETUP
```

최초 commit ID 앞부분:

```
de96b45
```

현재 프로젝트의 최초 세이프 포인트다.

26. GitHub

GitHub 사용자명:

```
UBshonen
```

Repository:


```
pixel-memories
```

공개 범위:

```
Public
```

GitHub Repository 생성 완료.

27. Remote 설정

등록된 remote:

```
origin
```

연결 대상:

```
UBshonen/pixel-memories
```

등록 명령:

```
git remote add origin https://github.com/UBshonen/pixel-memories.git
```

확인:

```
git remote -v
```

정상 연결 확인 완료.

28. 최초 Push

실행:

```
git push -u origin main
```

정상 성공.

결과:

```
[new branch] main -> main  
branch 'main' set up to track 'origin/main'.
```

따라서 현재:

```
Local main  
  ↓  
origin/main
```

추적 관계가 설정되어 있다.

앞으로 일반적인 push는:

```
git push
```

만 사용하면 된다.

29. 현재 Git 개념 학습 상태

현재까지 이해한 개념:

Repository

Git이 변경 이력을 관리하는 프로젝트 공간.

Commit

프로젝트의 세이프 포인트.

Branch

개발 이력이 뿔어나가는 작업 줄기.

현재 기본 브랜치:

```
main
```

Staging

다음 commit에 포함할 변경사항을 준비하는 단계.

```
git add
```

사용.

Remote

내 PC가 아닌 외부 Git repository.

현재:

```
origin = GitHub pixel-memories
```

Push

```
Local → GitHub
```

Pull

```
GitHub → Local
```

30. 앞으로 사용할 기본 Git 사이클

개발 중 기본적으로:

```
코드 수정
↓
git status
↓
git add .
↓
git commit -m "변경내용"
↓
git push
```

를 반복한다.

단, 무조건 `git add .` 을 사용하기보다는 Git에 익숙해지면서 어떤 파일을 staging하는지도 확인하는 습관을 만든다.

31. 현재 완료 목록

- [x] Pixel Memories 기본 컨셉 수립
- [x] 모바일 웹 방향 결정
- [x] Next.js 선택
- [x] React 프로젝트 생성
- [x] TypeScript 환경 생성
- [x] Tailwind CSS 환경 생성
- [x] Next.js 개발 서버 실행
- [x] localhost 접속 확인
- [x] Git 설치 및 동작 확인
- [x] Git repository 초기화
- [x] Git 기본 개념 학습
- [x] main branch 설정
- [x] 최초 staging
- [x] Git 사용자 정보 설정
- [x] 최초 commit
- [x] GitHub repository 생성
- [x] remote origin 연결
- [x] GitHub 인증
- [x] 최초 push
- [x] local main ↔ origin/main tracking 설정
- [] Phaser 설치
- [] Next.js + Phaser 연결
- [] Phaser Canvas 표시
- [] Map 구현
- [] Player 구현
- [] Player Movement 구현

32. 바로 다음 작업

다음 작업은:

Phaser가 무엇인지 이해하고 프로젝트에 설치한다.

진행 순서:

1. Phaser 개념
2. React / Next.js / Phaser 관계
3. `npm install phaser`
4. GameCanvas 생성
5. Phaser Game 생성
6. Next.js 페이지에서 GameCanvas 렌더링
7. `localhost:3000` 확인

- 8. commit
- 9. push

첫 번째 기술적 목표:

localhost:3000에서 Phaser Canvas가 정상적으로 표시되는 것

이다.

아직 캐릭터나 예쁜 맵부터 만들지 않는다.

33. 다음 작업에서 설명해야 할 핵심 개념

다음 개발을 진행하기 전에 최소한 다음 구조를 설명한다.

```
Browser
|
Next.js
|
React Component
|
GameCanvas
|
Phaser.Game
|
Scene
|
Canvas
```

각 계층이 왜 존재하는지 사용자가 이해한 후 구현한다.

특히:

React가 있는데 왜 Phaser가 필요한가?

Phaser가 Canvas를 만든다는 것은 무슨 뜻인가?

React Component 안에서 Phaser를 어떻게 실행하는가?

를 설명한다.

34. AI 작업 지침

이 프로젝트를 이어받은 AI는 다음 원칙을 지킨다.

사용자는 기존 개발 경험은 있지만:

```
React
Next.js
Phaser
Git
```

은 처음 사용한다.

따라서 명령어 또는 코드를 바로 던지는 방식으로 진행하지 않는다.

새 개념이 등장하면:

- ① 무엇인가
- ② 왜 필요한가
- ③ 기존 기술과 비교하면 무엇인가
- ④ Pixel Memories에서 무슨 역할인가
- ⑤ 실제 구현

순서로 설명한다.

예를 들어:

```
npm install phaser
```

를 제시하기 전에 최소한:

```
npm이 무엇을 하는지
package.json과 어떤 관계인지
node_modules에는 무엇이 들어가는지
phaser 패키지를 설치한다는 것이 실제로 무슨 의미인지
```

를 필요한 수준에서 설명한다.

단, 모든 내부 구현을 과도하게 파고들지는 않는다.

목표는:

AI 없이 모든 코드를 작성하는 것

이 아니라:

AI가 작성한 코드라도 사용자가 전체 구조와 실행 원리를 이해하는 것

이다.

35. 문서 유지 규칙

이 문서는 프로젝트가 끝날 때까지 유지한다.

PART A 수정 시점

다음과 같은 경우에만 수정한다.

프로젝트 방향 변경
핵심 기능 변경
아키텍처 변경
기술 스택 변경
MVP 범위 변경

즉 PART A는 프로젝트의 기준점이다.

PART B 수정 시점

개발 세션이 끝날 때마다 갱신한다.

특히:

현재 구현 상태
마지막 commit
현재 branch
새로 추가된 파일
새로 배운 개념
해결하지 못한 문제
다음 작업

을 업데이트한다.

36. 새 채팅 시작 방법

새 채팅에 이 문서를 제공한 후 다음과 같이 시작하면 된다.

Pixel Memories 개발 이어서 하자. MASTER 문서 기준으로 현재 DEVELOPMENT HANDOFF부터 확인하고, 다음 작업부터 진행하자. 새로운 기술이나 명령어가 나오면 원리를 설명하면서 한 단계씩 진행해줘.

그러면 현재 기준 다음 작업은:

PHASE 1 — Phaser 도입

부터 시작한다.

CURRENT CHECKPOINT

PIXEL MEMORIES

PROJECT

- └ Planning ✓
- └ Architecture Draft ✓
- |

DEVELOPMENT

- └ Next.js Setup ✓
- └ Local Dev Server ✓
- └ Git Init ✓
- └ First Commit ✓
- └ GitHub Repository ✓
- └ Remote ✓
- └ First Push ✓
- |
- └ Phaser Integration ← NEXT

Last known commit

de96b45
260813_PROJECT_SETUP

Next Objective

Next.js
↓
React
↓
Phaser
↓
Canvas 표시